

Exercícios — Módulo 6: Funções e Arrays

Aula 6 · 21/07 · Jornada DEV START — Programa START (TOTVS Paulista) Prazo: até **23/07** — é também a **entrega consolidada dos Fundamentos** (aulas 1–6). **Onde entregar:** na **atividade do Módulo 6 no Google Classroom**. Suba o código no seu **GitHub** e cole o **link do repositório/commit** na atividade. Depois, comemore postando um print no Discord [#conquistas](#) . 🎉🏁

Antes de começar

- Todos os exercícios são em **Harbour puro**, no VSCode.
- Fluxo de sempre: escrever → `hbm2 arquivo.prg` → `arquivo.exe` .
- Esta lista **funde** a prática de **funções** (Parte A) e de **arrays** (Parte B), e fecha com um desafio que junta os dois — a **dupla poderosa** função + array + loop.
- Organize cada exercício em funções pequenas e com nomes descritivos. 🧱
- Lembre: o índice de array **começa em 1**. 🕒
- Não trave sozinho(a): dúvida é no [#duvidas-parte1-harbour](#) . 🙌

Parte A — Funções (modularização) 🧱

Exercício 1 — Função ou procedimento? (conceitual)

Para cada item abaixo, responda se é uma **função** (retorna um valor) ou um **procedimento** (só executa uma ação) e justifique em uma frase:

- `Str()` — converte número em string
- `QOut()` — exibe texto na tela
- `Date()` — retorna a data atual
- `Len()` — retorna o tamanho de uma string ou array
- `Upper()` — retorna a string em maiúsculas
- `ClearScreen()` — limpa a tela



Entrega: responda no campo de texto da atividade no Classroom.

Exercício 2 — Relógio digital modular

Crie um “relógio digital” que exiba a hora no formato `HH:MM:SS`, atualizando por ~30 segundos. Organize em **funções separadas por responsabilidade**:

- `ObterHora()` — retorna a hora atual
- `FormatarHora(cHora)` — devolve a string formatada `HH:MM:SS`
- `ExibirHora(cHora)` — exibe na tela

Dica: a `Main()` deve apenas orquestrar (loop chamando as três funções).

Exercício 3 — Refatorando a calculadora

Pegue a calculadora do **Módulo 4** (tudo em `Main()`) e **refatore** usando funções separadas: - leitura dos valores (`LerNumero`) - uma função por operação (ou uma `Calcular` com `DO CASE`) - exibição do resultado (`MostrarResultado`)

Trate a **divisão por zero** devolvendo `.F.` da função de cálculo.

Meta: ao final, a `Main()` deve “se ler como uma frase”.

Exercício 4 — Biblioteca `matematica.prg`

Crie uma **biblioteca** separada `matematica.prg` com as funções abaixo, e um `principal.prg` que a inclua com `SET PROCEDURE TO matematica.prg` e teste cada uma:

- `FatorialN(nN)` — calcula o fatorial de N
- `EhPrimo(nN)` — retorna `.T.` se N é primo
- `MMC(nA, nB)` — mínimo múltiplo comum
- `MDC(nA, nB)` — máximo divisor comum

Dica: cada função é um bloco de Lego reutilizável — teste uma de cada vez.

Exercício 5 (Desafio) — Jokenpo modular

Crie um **Jokenpo** (pedra, papel, tesoura) usando **pelo menos 4 funções distintas**:

- `SortearJogadaCPU()` — sorteia a jogada do computador
- `ValidarJogada(cJogada)` — valida a entrada do usuário
- `DefinirVencedor(cJogada1, cJogada2)` — retorna quem ganhou

- `Main()` — orquestra o jogo com um loop de partidas
-

Parte B — Arrays (estruturas de dados) 🍬

Exercício 6 — Dias da semana por índice

Crie um programa que: - Defina um array literal com os 7 dias da semana - Leia um número de **1 a 7** e exiba o nome do dia correspondente - **Valide** se o número está no intervalo (senão, avise o usuário)

Lembre: `aDias[1]` é o primeiro dia (índice começa em 1).

Exercício 7 — Estatísticas de 10 números

Leia 10 números, armazene em um array e exiba: - Os números em **ordem crescente** (use `ASORT`) - A **soma** e a **média** - O **maior** e o **menor** (dica: após `ASORT`, são `aV[1]` e `aV[Len(aV)]`)

Exercício 8 — Sistema de notas (array multidimensional)

Implemente um sistema de notas: - Leia o **nome** e **4 notas** de **N alunos** (N definido pelo usuário) - Armazene em um **array multidimensional** (cada linha: `{nome, n1, n2, n3, n4}`) - Calcule a **média** de cada aluno - Liste os **aprovados** (média ≥ 7) e os **reprovados**

Exercício 9 — Mini-carrinho de compras

Crie um mini-carrinho: - Array de produtos, cada item `{nome, preço}` - Loop para **adicionar** itens ao carrinho (encerra quando o usuário quiser) - Exiba a **lista de itens** e o **total** ao final

Exercício 10 (Desafio) — Bubble Sort na mão

Implemente o **Bubble Sort manualmente**, sem usar `ASORT`: - Crie uma função `BubbleSort(aVetor)` que ordene o array em ordem crescente - Teste com um array de 10 números (aleatórios ou digitados)

Dica: dois loops encaixados; troque elementos vizinhos fora de ordem usando uma variável temporária.

Parte C — A dupla poderosa (fusão) 💪

Exercício 11 — Mini-controle de estoque (integrador)

Junte **tudo**: funções + array de registros + loop. Faça um **Controle de Estoque Simplificado** (só em memória, sem banco):

- Array de produtos, cada item: `{codigo, nome, quantidade, preco_unitario}`
- **Menu** em loop com as opções:
 - **1** — Cadastrar produto (usa `AADD`)
 - **2** — Listar produtos (percorre com `FOR`)
 - **3** — Entrada de estoque (aumenta a quantidade)
 - **4** — Saída de estoque (diminui; **valide** estoque suficiente)
 - **5** — Buscar produto por código (função que devolve a posição, ou 0)
 - **6** — Relatório: valor em estoque por produto (`qtd × preço`) e **total geral**
 - **0** — Sair

Requisitos: - **Uma função por funcionalidade** (menu, cadastrar, listar, entrada, saída, buscar, relatório) - **Valide** todas as entradas - Coloque as operações de array numa **biblioteca separada** (ex.: `estoque_lib.prg`)

Este exercício é a prova de que você fechou os Fundamentos: é o padrão **coleção + loop + funções** que sustenta qualquer sistema real — inclusive o Protheus. 🏰

Como sei que acertei?

Exercício	Está pronto quando...
1	Você classificou cada item como função/procedimento com justificativa
2	A hora aparece em <code>HH:MM:SS</code> e a <code>Main()</code> só chama as 3 funções
3	A calculadora roda igual à antiga, mas dividida em funções (e trata ÷ por zero)
4	<code>matematica.prg</code> é incluída com <code>SET PROCEDURE TO</code> e as 4 funções devolvem valores corretos
5	O Jokenpo joga contra a CPU, valida a entrada e aponta o vencedor (≥ 4 funções)
6	Digitar 1–7 mostra o dia certo; fora do intervalo, avisa
7	Os 10 números saem ordenados, com soma, média, maior e menor corretos
8	Cada aluno tem sua média e a lista separa aprovados de reprovados
9	Os itens são listados e o total bate com a soma dos preços
10	<code>BubbleSort()</code> ordena sem usar <code>ASORT</code>
11	O menu roda em loop, todas as opções funcionam e o relatório soma o total geral

✅ Fez de 1 a 9? Você cumpriu o essencial do módulo (funções e arrays). O 10 e o 11 são os “algo a mais” que consolidam a **dupla poderosa** — e fecham os Fundamentos com chave de ouro. 🏆



Como organizar no seu repositório

Salve os exercícios desta aula na pasta `modulo-06/`, um arquivo por exercício (guia completo em `pre-curso/setup-github.md`):

```

modulo-06/
├─ ex01-funcao-ou-procedimento.md
├─ ex02-relogio-modular.prg
├─ ex03-calculadora-refatorada.prg
├─ ex04-biblioteca-matematica/      (vários arquivos)
├─ ex05-jokenpo-modular.prg         (desafio – opcional)
├─ ex06-dias-da-semana.prg
├─ ex07-estatisticas-numeros.prg
├─ ex08-sistema-notas.prg
├─ ex09-carrinho-compras.prg
├─ ex10-bubble-sort.prg             (desafio – opcional)
└─ ex11-controle-estoque/          (vários arquivos, desafio – opcional)

```

Depois: `git add .` → `git commit -m "modulo 06 – exercicios"` → `git push`, e cole o link da pasta na atividade do Módulo 6 no Classroom.

Guia de referência rápida

```

// ----- FUNÇÕES -----
FUNCTION Nome(p1, p2) ... RETURN valor    // função (retorna algo)
FUNCTION Nome(...)    ... RETURN NIL      // procedimento (não retorna)
hb_Default(@cParam, "padrao")            // valor padrão de parâmetro opcional
Funcao(@nVar)                             // @ = passagem por REFERÊNCIA
SET PROCEDURE TO biblioteca.prg           // inclui biblioteca de funções
// Escopos: LOCAL · PRIVATE (aqui+filhas) · PUBLIC · STATIC

// ----- ARRAYS (índice começa em 1) -----
LOCAL aV := {}                          // vazio      | LOCAL aV := {"a","b"} // literal
LOCAL aV := Array(5)                    // 5 posições NIL
aV[1]                                     // acessa      | aV[2] := "x"           // modifica
Len(aV)                                  // tamanho
AADD(aV, x)                              // adiciona no final
ASIZE(aV, n) · ADEL(aV, n)                // redimensiona · deleta posição
ASORT(aV)                                 // ordena crescente
ASCAN(aV, x)                              // busca (posição; 0 = não achou)
AEVAL(aV, {|e| ... })                    // executa bloco p/ cada elemento
aMatriz[i][j]                             // multidimensional (linha, coluna)

// ----- A DUPLA PODEROSA -----
FOR nI := 1 TO Len(aV)                   // loop percorre o array...
    Processar(aV[nI])                     // ...e a função processa cada item
NEXT

// Úteis: Val(cTxt) Str(nNum,tam,dec) Upper() AllTrim() Max() Min()

```



Documentação do Harbour: <https://harbour.github.io/doc/>